

Lab 21 - Banco de Dados

Essas notas foram criadas com o intuito de auxiliar o estudo da disciplina de Programação Web, do Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas do Instituto Federal de São Paulo, Campus Boituva. Para qualquer erro ou omissão, e-mail de contato: anisio.silva@ifsp.edu.br

Repositório GitHub: [GitHub](#)

1 Métodos Assíncronos e Integração com MySQL

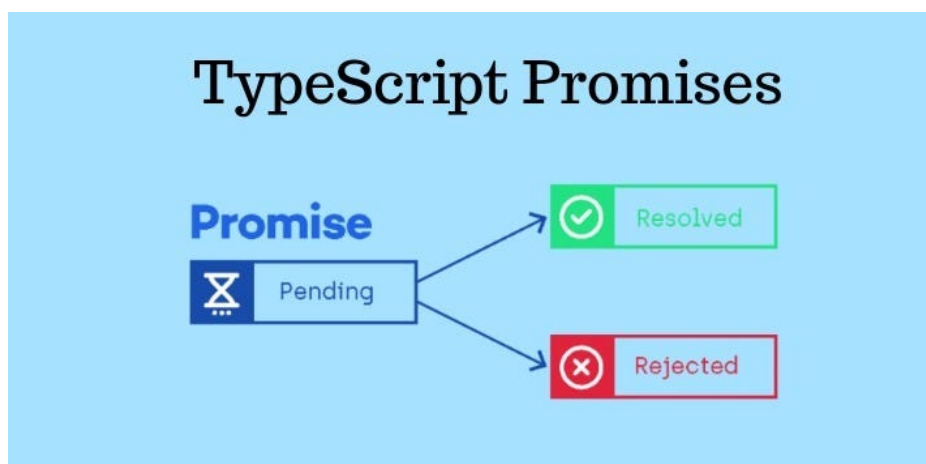
Nesta aula, abordaremos o uso de métodos assíncronos em TypeScript, com foco especial na integração com o banco de dados MySQL. Entenderemos o funcionamento de **promises** e como utilizá-las para controlar operações que ocorrem de forma assíncrona, como conexões com banco de dados.

1.1 Promises

Em TypeScript (e também em JavaScript), uma **Promise** é um objeto que representa o eventual sucesso ou fracasso de uma operação assíncrona. Ela pode estar em três estados:

- **Pendente (pending)**: Aguardando a conclusão da operação.
- **Resolvida (resolved)**: A operação foi concluída com sucesso.
- **Rejeitada (rejected)**: A operação falhou.

Figura 1: Funcionamento de uma *Promise*



Veja o exemplo a seguir:

```
1 function delay(ms: number): Promise<void> {  
2   return new Promise<void>((resolve, reject) => {  
3     setTimeout(() => {  
4       if (ms > 2000) {  
5         console.log("Operação concluída após " + ms / 1000 + " segundos.");  
6         resolve();  
7       } else {
```

```
8         console.log("Tempo insuficiente para concluir a operação.");
9         reject();
10    }
11    }, ms);
12  });
13 }
14
15 async function test() {
16   try {
17     console.log("Antes da execução do delay...");
18     await delay(2000);
19     console.log("Conclusão da promessa.");
20   } catch (error) {
21     console.error("Erro:", error);
22   } finally {
23     console.log("Depois da execução do delay...");
24   }
25 }
26
27 test();
```

Note que o uso de `async` e `await` permite que o código assíncrono pareça síncrono, facilitando a leitura e o controle do fluxo de execução.

2 Utilizando Banco de Dados MySQL

Agora que sabemos trabalhar como promises, vamos aprender como conectar a um banco de dados MySQL e realizar as principais operações de um CRUD (*Create, Read, Update, Delete*).

Primeiramente, crie um banco de dados chamado Vendas:

Listing 1: Schema Vendas

```
CREATE SCHEMA aula21;
```

Utilizaremos uma estrutura mais abstrata, conforme ilustrado na figura a seguir:

```
aula_21/
|- src/
|   |- app.ts
|   |- model/
|       |- Product.ts
|   |- repository/
|       |- ProductRepository.ts
|   |- controller/
|       |- ProductController.ts
|   |- service/
|       |- ProductService.ts
|   |- database/
|       |- mysql.ts
|   |- routes/
|       |- router.ts
|- node_modules/
```

```
|  
|- tsconfig.json  
|- package-lock.json
```

Nessa estrutura, estão sendo utilizadas duas novas camadas. A primeira, “*database*”, será responsável pelas configurações referentes à conexão com o banco.

Da mesma forma, a camada “*routes*” introduz uma abordagem arquitetural nova e fundamental para o desacoplamento do projeto. Em aplicações iniciais, é comum concentrar a definição dos caminhos (*endpoints*) e a lógica de resposta diretamente no arquivo principal da aplicação (*app.ts*). Contudo, à medida que o sistema expande, essa prática compromete a manutenibilidade do código.

O arquivo *router.ts* resolve esse problema ao centralizar o mapeamento das rotas HTTP (como GET e POST) e delegar a execução das requisições para os respectivos controladores (*Controllers*).

A descentralização das responsabilidades visa mitigar o acoplamento de código. O arquivo *mysql.ts*, contido no diretório “*database*”, encapsula a criação da conexão (*Connection Pool*) e expõe uma função genérica para a execução de instruções SQL. Adicionalmente, esta camada assume o papel de *bootstrap* da infraestrutura, disparando rotinas de inicialização que garantem a existência prévia das tabelas necessárias antes que o servidor HTTP comece a escutar as requisições na porta configurada.

O fluxo de dados segue uma hierarquia bem definida em direção à persistência:

- **Routes (*router.ts*):** Atua como a porta de entrada da aplicação, interceptando os *endpoints* HTTP e direcionando o fluxo para as funções de controle.

```
1  import { Router, Request, Response } from 'express';  
2  import { ProductController } from '../controller/ProductController';  
3  
4  const router = Router();  
5  const controller = new ProductController();  
6  
7  // Products  
8  router.post('/products', (req: Request, res: Response) => { controller.criar(  
9    req, res); });  
10 router.get('/products', (req: Request, res: Response) => { controller.listar(  
11   req, res); });  
12  
13 export default router;
```

Listing 2: Router.ts

- **Model (*Product.ts*):** Define a representação estrutural da entidade de negócio no ecossistema TypeScript, utilizando recursos de tipagem estrita para garantir a integridade do dado.
- **Repository (*ProductRepository.ts*):** Concentra exclusivamente as operações de manipulação de dados (comandos *Data Manipulation Language* – DML, como INSERT e SELECT). Ele abstrai as consultas brutas da camada de aplicação utilizando o padrão *Singleton*.

Ao isolar a criação das tabelas e a configuração de conectividade em “*database*”, o repositório permanece coeso, eliminando redundâncias à medida que novas entidades e tabelas forem acopladas ao sistema web.

Para acessar o banco de dados será necessário instalar o cliente MySQL2 utilizando o comando a seguir:

Listing 3: Install mysql2

```
npm install mysql2
```

2.1 Configurando a conexão com o banco de dados

Vamos configurar o arquivo `mysql.ts` da seguinte forma:

```
1 import mysql, { Connection, QueryError } from 'mysql2';
2
3 const dbConfig = {
4   host: 'localhost',
5   port: 3306,
6   user: 'root',
7   password: 'mysql',
8   database: 'aula21'
9 };
10
11 const mysqlConnection: Connection = mysql.createConnection(dbConfig);
12
13 mysqlConnection.connect((err) => {
14   if (err) {
15     console.error('Erro ao conectar ao banco de dados:', err);
16     throw err;
17   }
18   console.log('Conexao bem-sucedida com o banco de dados MySQL');
19 });
```

Após configurar o arquivo, compile o projeto e execute o arquivo `mysql.js` presente na pasta `dist`. O resultado esperado no terminal é:

```
Conexao bem-sucedida com o banco de dados MySQL
```

2.2 Criando uma tabela no banco

Para começar, vamos criar a entidade que irá representar um produto. Crie a classe `Product.ts` na pasta `model`, conforme o código abaixo:

```
1 export class Product {
2   constructor(
3     public id: number | null,
4     public name: string,
5     public price: number
6   ) {}
7 }
```

Listing 4: Classe Product

A estrutura adotada para a classe `Product` introduz dois conceitos fundamentais do TypeScript que otimizam o desenvolvimento de aplicações *backend*: as propriedades de parâmetro (*Parameter Properties*) e os tipos de união (*Union Types*).

2.2.1 A Vantagem da Estrutura Abstrata e Compacta

O construtor da classe utiliza modificadores de acesso diretamente na sua assinatura (neste caso, `public`). Essa sintaxe reduz drasticamente o código repetitivo (*boilerplate*) tradicionalmente encontrado em linguagens como Java ou C#.

Ao declarar `public name: string`, o compilador do TypeScript realiza três ações automaticamente por debaixo dos panos:

1. Declara o atributo na classe;
2. Recebe o parâmetro no construtor;
3. Atribui o valor implicitamente utilizando o mapeamento `this.name = name`.

Essa abordagem torna os modelos de dados muito mais limpos, legíveis e fáceis de manter, permitindo que o desenvolvedor foque na estrutura da entidade e não em regras repetitivas de inicialização de propriedades.

2.2.2 Por que utilizar o Union Type (`number | null`) no ID?

O uso do caractere `|` define um *Union Type* (Tipo de União), indicando que a propriedade `id` pode assumir o tipo `number` OU o valor `null`. Essa escolha é crucial quando integramos a aplicação a um banco de dados relacional com chaves primárias do tipo `AUTO_INCREMENT`:

- **Cenário de Inserção (Antes do Banco):** Quando a aplicação recebe os dados de um novo produto vindo de um formulário *web* (ex: Nome e Preço), esse registro ainda não existe no banco de dados. Portanto, ele não possui um identificador. O valor `null` serve justamente para representar de forma explícita essa **ausência inicial de ID**.
- **Cenário de Persistência (Pós-Banco):** Ao enviar o objeto para o repositório, o MySQL executa o `INSERT` e gera um ID numérico automaticamente. O repositório, então, captura esse valor e devolve uma nova instância da classe onde o `id` passa a ser um `number` legítimo.

Dessa forma, a tipagem `number | null` protege o código em tempo de compilação, forçando o desenvolvedor a lembrar que o ID pode não estar disponível em determinados momentos do ciclo de vida do dado, evitando erros em tempo de execução (*runtime errors*).

2.2.3 Criando um método universal de execução de query

Agora que temos nossa entidade definida, precisamos criar uma tabela no banco de dados que a represente. Para facilitar, vamos alterar o arquivo de configuração `mysql.ts`. Adicione a seguinte função ao arquivo:

```
1 export function executarComandoSQL(query: string, valores: any[]): Promise<any> {
2   return new Promise<any>((resolve, reject) => {
3     mysqlConnection.query(query, valores, (err, resultado) => {
4       if (err) {
5         console.error('Erro ao executar a query.', err);
6         reject(err);
7       }
8       resolve(resultado);
9     });
10  });
11 }
```

Suponha que queremos fazer uma inserção de um novo produto na tabela do banco de dados. A query SQL seria algo como:

Listing 5: Insert

```
INSERT INTO produtos (nome, preco) VALUES ('Camiseta', 20.99);
```

Para executar a mesma operação no TypeScript precisamos fazer algumas mudanças:

```
1  const query = "INSERT INTO produtos (nome, preco) VALUES (?, ?)";
2  const valores = ["Camiseta", 20.99];
3
4  executarComandoSQL(query, valores);
```

Os pontos de interrogação indicam onde os valores serão substituídos na query. Agora, ao utilizar essa query na função `executarComandoSQL`, você precisa fornecer os valores que serão inseridos no lugar dos pontos de interrogação. Isso é feito passando um array de valores como segundo argumento para a função.

Dentro da função `executarComandoSQL`, os valores no array serão substituídos nos lugares dos pontos de interrogação na query SQL antes de executar a operação no banco de dados. Isso permite que você construa consultas dinâmicas e seguras contra injeções de SQL, já que os valores são tratados como dados e não como parte da própria consulta.

2.2.4 Criando um Repository Adaptado

Com a função `executarComandoSQL` devidamente estruturada, podemos avançar para o isolamento da lógica de persistência. Para isso, criaremos a classe `ProductRepository` na pasta `repository`. Esta camada será a única responsável por gerenciar todo o acesso ao banco de dados relacionado à entidade `Product`.

Abaixo, apresentamos a estrutura inicial da classe, focada na definição do seu ciclo de vida e na exportação da *query* de criação da tabela:

```
1  import { executarComandoSQL } from "../database/mysql";
2  import { Product } from "../model/Product";
3
4  export class ProductRepository {
5      private static instance: ProductRepository;
6
7      private constructor() {}
8
9      static getInstance(): ProductRepository {
10         if (!this.instance) {
11             this.instance = new ProductRepository();
12         }
13         return this.instance;
14     }
15
16     static getCreateTableQuery(): string {
17         return `
18         CREATE TABLE IF NOT EXISTS Product (
19             id INT AUTO_INCREMENT PRIMARY KEY,
20             name VARCHAR(255) NOT NULL,
21             price DECIMAL(10,2) NOT NULL
22         );
23     `;
```

```
24 }  
25 }
```

Listing 6: Classe ProductRepository

Uma vez definida a query de criação da tabela, precisamos garantir que, ao iniciar a aplicação, a tabela já exista no banco de dados. Em outras palavras, a aplicação deve estar alinhada ao schema do banco. Para isso, vamos adicionar a função ‘inicializarBanco’ no arquivo ‘mysql.ts’:

```
1  export async function inicializarBanco(): Promise<void> {  
2    console.log('Sincronizando schemas do banco de dados...');  
3  
4    const schemas = [  
5      ProductRepository.getCreateTableQuery(),  
6      // Novas tabelas aqui ...  
7    ];  
8  
9    try {  
10  
11      await executarComandoSQL(`USE ${dbConfig.database}`, []);  
12      console.log(`Conectado ao schema: ${dbConfig.database}`);  
13  
14      for (const query of schemas) {  
15        await executarComandoSQL(query, []);  
16      }  
17      console.log('Todos os repositórios foram inicializados com sucesso.');
```

2.2.5 Inserção de dados na tabela

Vamos fazer nossa primeira inserção:

```
1  async insertProduct(product: Product): Promise<Product> {  
2    const resultado = await executarComandoSQL(  
3      "INSERT INTO product (name, price) VALUES (?, ?)",  
4      [product.name, product.price]  
5    );  
6  
7    const idGerado = resultado.insertId;  
8  
9    const newProduct = new Product(idGerado, product.name, product.price);  
10  
11    console.log('Produto inserido com sucesso:', newProduct);  
12    return newProduct;  
13  }
```

Como não foi lançada nenhuma exceção, podemos entender que nosso código obteve êxito, porém a mensagem dentro do método createTable está sendo executada antes da resolução da função executarComandoSQL, na próxima aula focaremos na solução desse problema.

2.3 Integração com a camada Service

O método createProduct é declarado como async, o que significa que qualquer valor retornado será automaticamente encapsulado em uma Promise. Como o método insertProduct da camada repository

já retorna uma **Promise** e, no exemplo a seguir, não existe uma validação adicional sobre o resultado, não há necessidade de utilizar **await** antes do **return**.

```
1 import { Product } from "../model/Product";
2 import { ProductRepository } from "../repository/ProductRepository";
3
4 export class ProductService {
5     private productRepository = ProductRepository.getInstance();
6
7     async createProduct(data: any): Promise<Product> {
8         if (!data.name || !data.price) {
9             throw new Error('Favor informar os campos obrigatórios');
10        }
11
12        const novoProduct = new Product(null, data.name, Number(data.price));
13
14        return this.productRepository.insertProduct(novoProduct);
15    }
```

Listing 7: ProductService

Se fosse utilizado **await**, o valor da **Promise** seria resolvido imediatamente e retornado como um objeto **Product**, para em seguida ser novamente empacotado em uma **Promise** pelo compilador. Esse processo é redundante e nesse caso em específico, não traz benefícios, apenas adiciona uma etapa desnecessária.

Portanto, o retorno é simplesmente:

```
1 return this.productRepository.insertProduct(novoProduct);
```

Assim, o método **createProduct** propaga a **Promise<Product>** diretamente para o chamador, mantendo o fluxo assíncrono mais eficiente e consistente.

2.4 Integração com a camada Controller

No *controller* teremos a resolução da **Promise** que foi propagada desde a camada *database*, passando por *repository* e *service*. É nesta camada que o resultado assíncrono é finalmente convertido em uma resposta HTTP para o cliente. Além disso, o *controller* assume a responsabilidade de validar os dados recebidos na requisição, tratar erros e definir o código de status apropriado para cada situação (por exemplo, 400 para requisições inválidas, 201 para criação bem-sucedida e 500 para falhas internas).

```
1 import { Request, Response } from 'express';
2 import { ProductService } from '../service/ProductService';
3
4 export class ProductController {
5     private productService = new ProductService();
6
7     async criar(req: Request, res: Response) {
8         try {
9             const { name, price } = req.body;
10
11             if (!name || !price) {
12                 return res.status(400).json({ erro: "Nome e preço são obrigatórios." });
13             }
14
15             const novoProduto = await this.productService.createProduct({ name, price });
16             return res.status(201).json(novoProduto);
17         } catch (error) {
18             return res.status(500).json({ erro: "Erro interno ao salvar produto." });
19         }
20     }
```


21 }

Listing 8: ProductController

Dessa forma, o *controller* atua como a interface entre o cliente e a aplicação, garantindo que toda a cadeia assíncrona (do *database* até o *service*) seja resolvida e exposta de maneira segura e padronizada.

2.5 Integração com o app.ts

O arquivo `app.ts` ficou mais simplificado agora que consome o `router.ts`. Na função `startServer`, além de encapsular o método `listen`, também é realizada a chamada para `inicializarBanco`, responsável por garantir que o banco de dados esteja sincronizado antes que o servidor HTTP comece a escutar as requisições.

Essa abordagem traz duas vantagens principais:

- **Separação de responsabilidades:** o `router.ts` concentra as rotas da aplicação, enquanto o `app.ts` cuida apenas da configuração inicial e da execução do servidor.
- **Robustez na inicialização:** ao assegurar que as tabelas estejam criadas previamente, evitam-se erros de execução durante o atendimento das requisições.

```
1 import express from "express"
2 import router from "../routes/router";
3 import { inicializarBanco } from "../database/mysql";
4
5 const app = express()
6
7 const PORT = process.env.PORT ?? 3000
8 app.use(express.json())
9
10
11 app.use('/api', router);
12
13 async function startServer() {
14   await inicializarBanco();
15
16   app.listen(PORT, () => {
17     console.log(`Servidor rodando em http://localhost:${PORT}`);
18   });
19 }
20
21 startServer();
```

Listing 9: App.ts

2.6 Curiosidades sobre o retorno da biblioteca MySQL12

Ao executar uma instrução `SELECT`, a biblioteca retorna um `array` de objetos literais (JSON puro), sem qualquer associação direta às classes da aplicação. Por isso, é necessário realizar o mapeamento manual para instâncias da classe `Product`, como nos métodos `findAll` e `findById`. Esse processo garante que os dados obtidos do banco sejam convertidos em objetos de domínio, mantendo a consistência da arquitetura orientada a objetos.

```
1 async findAll(): Promise<Product[]> {
2   const linhas = await executarComandoSQL("SELECT id, name, price FROM product", []);
3 }
```

```
4 // O MySQL retorna um array de objetos puros. Mapeamos para objetos da classe Product.
5 const produtos: Product[] = linhas.map((linha: any) => {
6   return new Product(linha.id, linha.name, Number(linha.price));
7 });
8
9 return produtos;
10 }
11
12 async findById(id: number): Promise<Product | null> {
13   const linhas = await executarComandoSQL(
14     "SELECT id, name, price FROM product WHERE id = ?",
15     [id]
16   );
17
18   if (linhas.length === 0) {
19     return null;
20   }
21
22   const linha = linhas[0];
23   return new Product(linha.id, linha.name, Number(linha.price));
24 }
```

Listing 10: App.ts

Ao executar comandos como INSERT, UPDATE ou DELETE, a biblioteca não retorna os registros afetados, mas sim um objeto do tipo `ResultSetHeader`, contendo metadados da operação. Entre os campos mais relevantes estão:

- `affectedRows`: número de linhas afetadas pela instrução.
- `insertId`: identificador gerado automaticamente em operações de INSERT.
- `changedRows`: número de linhas efetivamente alteradas em um UPDATE.
- `warningStatus`: indica se houve algum aviso durante a execução.

Um exemplo de retorno pode ser observado no log abaixo:

```
1 Dentro callback ResultSetHeader {
2   fieldCount: 0,
3   affectedRows: 0,
4   insertId: 0,
5   info: '',
6   serverStatus: 2,
7   warningStatus: 0,
8   changedRows: 0
9 }
```

Esse comportamento explica por que, ao inserir um novo registro, precisamos capturar o campo `insertId` e reconstruir manualmente o objeto da entidade (`Product`, no caso). Assim, garantimos que o retorno para a camada superior seja uma instância completa e consistente da classe de domínio.

Exercícios

1. Implemente o método `deleteProduct`.
2. Implemente o método `updateProduct`.
3. Desenvolva uma API com as funcionalidades desenvolvidas.